A Project Report On

HTML QUICK-STYLER

Under Manonmaniam Sundaranar university

Submitted By

BAVITHRA DEVI	**23083300500112006**
GAYATHRI K	**23083300500112007**
GOBIKA CHANDRAN L	**23083300500112008**
HATHIJATHUL HASMIYA M	**23083300500112009**
JAMEELA DHEEZAAN M.S	**23083300500112011**

DEPARTMENT OF COMPUTER APPLICATIONS

ARUNACHALA ARTS & SCIENCE (WOMEN) COLLEGE

Manonmaniam Sundaranar University

Tirunelveli-627 001

HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

Project Description

HTML Quick-Styler is an intelligent web page generation platform designed to simplify and accelerate modern web development. The platform leverages **IBM's Granite 3.3 2B Instruct model** to generate well-structured HTML and professionally styled CSS based on user input. It enables users to quickly create responsive, visually appealing web pages without extensive manual coding.

The system addresses the challenge of rapid web development by providing **AI-powered HTML generation, automatic section detection, modern CSS styling, color palette management, and production-ready code output**. By analyzing user descriptions—such as page title, required sections, and color preferences—the platform automatically constructs a semantic HTML structure and applies modern design elements.

The platform is built with **Gradio's intuitive interface**, allowing users to interact with the system easily and generate customized web pages in a user-friendly environment. Through AI-powered analysis and sophisticated template generation, HTML Quick-Styler provides personalized, modern web page designs aligned with current design trends.

Overall, **HTML Quick-Styler transforms web development into an intelligent and efficient process**, enabling users to generate professional HTML pages quickly while maintaining high design and usability standards.

Scenarios

Scenario 1: Portfolio Website Generation

A freelance designer wants to quickly create a professional portfolio website. The system analyzes the designer's preferences and generates a complete site structure with a hero section, portfolio gallery, about section, client testimonials, and a contact form with validation. It also adds a professional footer with social media links, ensuring a responsive and polished design.

Scenario 2: SaaS Landing Page

A startup needs an effective landing page for a new product launch. The system creates a strong hero banner highlighting the product's value, a features section with icons and descriptions, and a pricing comparison table. It also includes testimonials for credibility and strategically placed call-to-action buttons to improve conversions.

Scenario 3: Corporate Website

A company requires a professional corporate website. The system generates a clean layout with a sticky navigation header, company overview section, services grid, and team member profiles. It also includes a contact section with an inquiry form and a professional footer with important links and policies.

Scenario 4: Custom Color Palette Exploration

A designer wants to explore different color schemes for a project. The system provides eight curated palettes such as modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal. Each palette includes hex color codes and real-time previews, ensuring consistent design and support for dark-theme backgrounds.

Architecture Overview

HTML Quick-Styler is built as a modular platform combining Python backend with IBM Granite 3.3 2B Instruct model for intelligent HTML generation. The architecture prioritizes user experience, design quality, and code production through Gradio's interactive interface and sophisticated template generation.

Core Technologies

- Python 3.x: Core programming language for application logic
- IBM Granite 3.3 2B Instruct: AI model for intelligent content generation

- Gradio: Interactive web interface framework for user interaction
- HTML5/CSS3: Modern web standards for page generation
- JavaScript: Client-side interactivity and animations
- Google Colab: Cloud-based development environment
- Responsive Design: Mobile-first approach with media queries

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

- Python Basics: Functions, classes, dictionaries, string manipulation
- HTML/CSS: Understanding of HTML structure and CSS styling
- Gradio Framework: Basic understanding of interface components
- Web Design: Color theory, responsive design principles
- Google Colab: Familiarity with notebook interface

Hardware Requirements

- Processor: Intel i3 or equivalent (Colab runs on cloud)
- RAM: Minimum 4GB local (Colab provides GPU/TPU)
- Storage: 1GB free space for generated files
- Display: Minimum 1280x720 resolution for interface

Project Workflow

Phase 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model

Activity 1.1: Set up Google Colab Environment

- Create new Google Colab notebook
- Enable GPU runtime for better performance
- Install required Python packages
- Verify environment configuration

Activity 1.2: Configure IBM Granite Model

- Access Hugging Face model repository
- Load Granite 3.3 2B Instruct model
- Test model inference capabilities
- Configure generation parameters

Activity 1.3: Install and Configure Gradio

- Install Gradio via pip
- Test basic Gradio interface
- Configure sharing options
- Verify interface rendering

Phase 2: Core Application Development

Build HTML generation engine and template system

Activity 2.1: Build Color Palette Generator

- Create ColorPaletteGenerator class
- Define 8 color palette schemes
- Implement palette selection logic
- Add color validation methods

Activity 2.2: Implement HTML Page Builder

- Create HTMLPageBuilder class
- Build section detection algorithm
- Implement template generation methods
- Create SEO meta tags generator

Activity 2.3: Develop CSS Generation Engine

- Build comprehensive CSS generator
- Implement responsive design rules
- Add animation and transition effects
- Create mobile-first media queries

Activity 2.4: Build Section Templates

- Create hero section template
- Build features grid template
- Implement portfolio gallery template
- Add contact form template
- Create pricing and testimonial templates

Phase 3: Gradio Interface Development

Create interactive user interface with multiple tabs

Activity 3.1: Interactive HTML Generation & Preview Interface

Activity 3 focuses on designing and implementing a **complete HTML generation and visualization interface** that enables users to create, customize, preview, and understand web page layouts through an intuitive, multi-section workflow.

Phase 4: Google Colab Deployment:

Integrate all components and perform comprehensive testing

Activity 4.1: End-to-End Integration

- Connect all interface components
- Test data flow between tabs
- Validate generation pipeline
- Test error handling

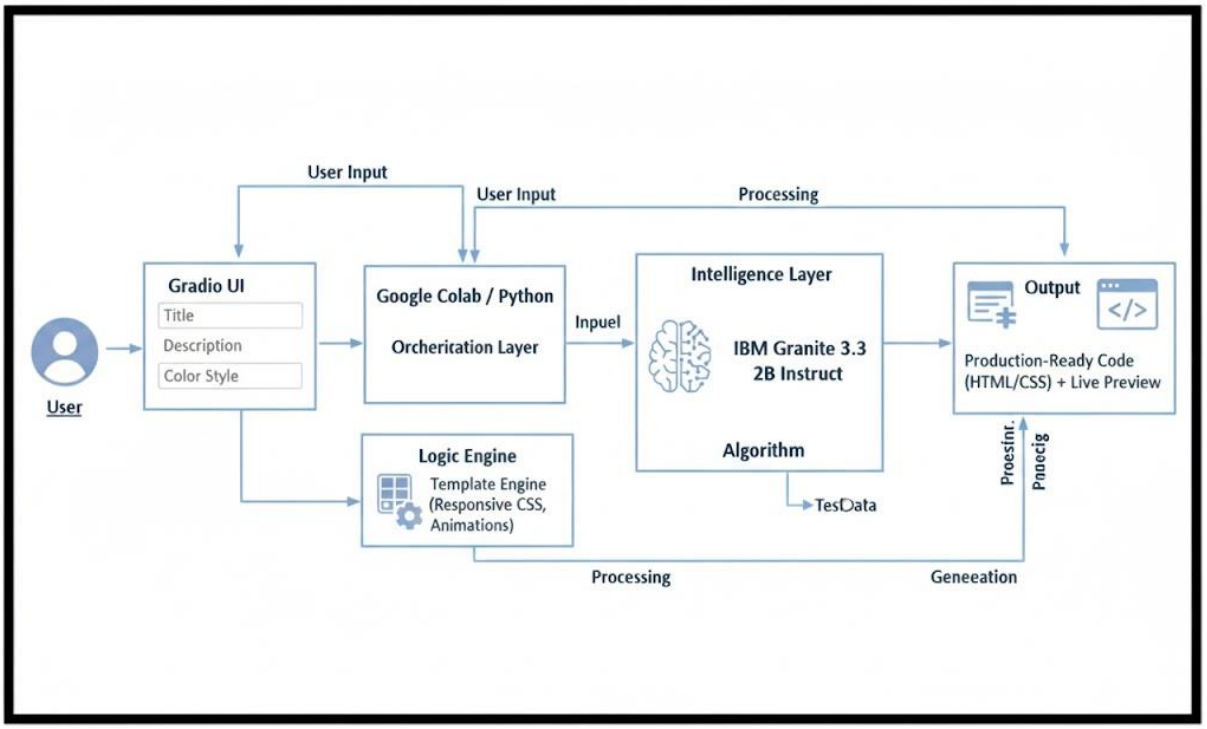
Phase 5: Deployment & Documentation

Deploy application and create comprehensive documentation

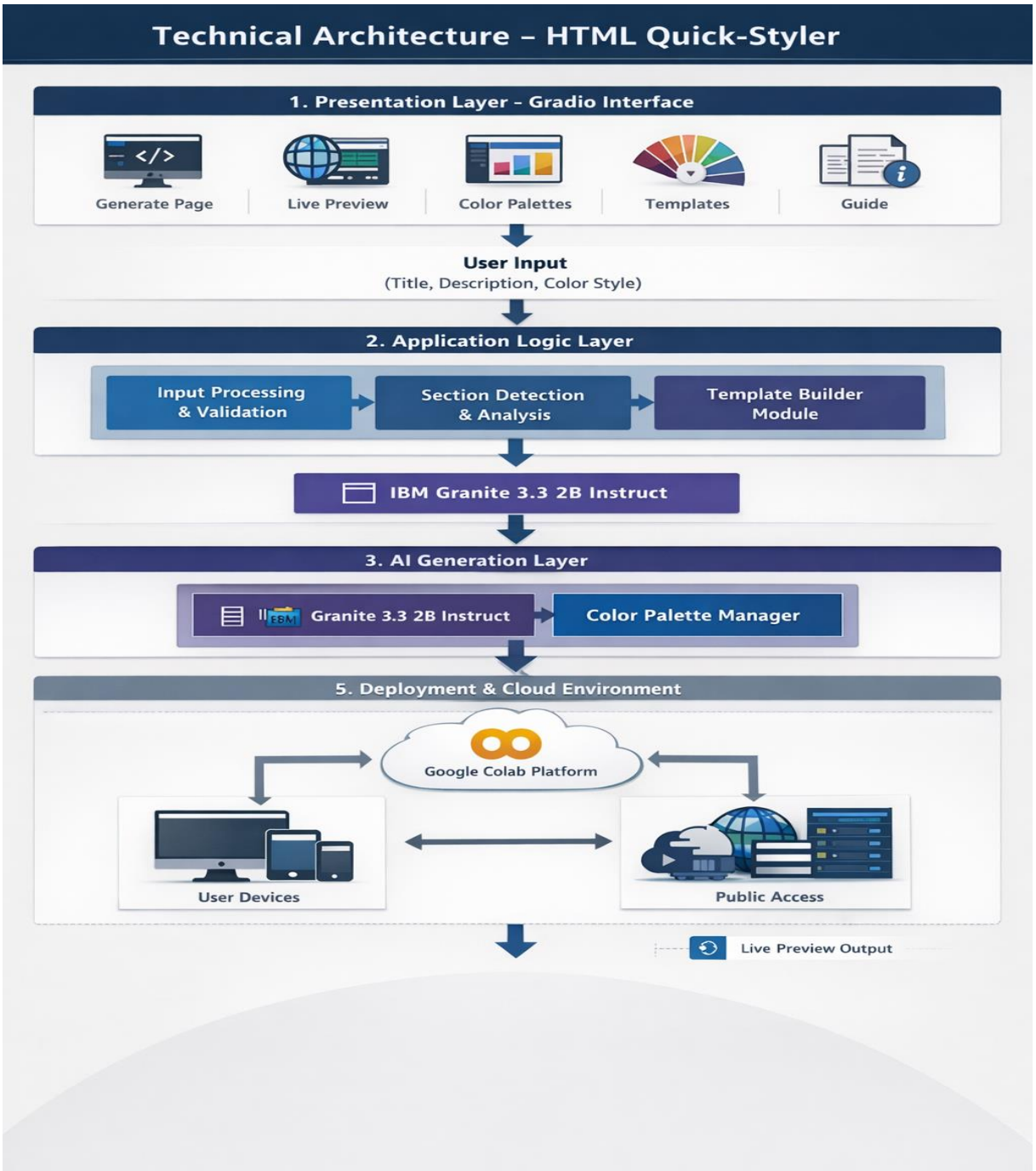
Activity 5.1: Functionality testing

- Configure Gradio sharing settings
- Enable public access link
- Test external access
- Document deployment steps

Technical Architecture



Project Structure:



MILESTONE 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model

Activity 1.1: Install Dependencies & Setup Environment

Create requirements.txt:

```
[ ]
!pip install -q gradio

import gradio as gr
import json
import re
from datetime import datetime
from typing import Tuple, Dict, List
import random
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 1.2: Configure IBM Granite Model

Access Hugging Face and load model:

Model: ibm-granite/granite-3.3-2b-instruct

Colab Notebook:

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
  messages,
  add_generation_prompt=True,
  tokenize=True,
  return_dict=True,
  return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))

Loading weights: 100% [=========================] 362/362 [00:04<00:00, 139.69it/s, Materializing param=model.norm.weight]
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 2: Core Application Development

Activity 2.1: Color Palette Generator Implementation

Create ColorPaletteGenerator class with 8 predefined color schemes:

The ColorPaletteGenerator class provides cohesive color palettes for different design styles including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal. Each palette

includes primary, secondary, accent, background, surface, text primary, text secondary, and border colors optimized for dark theme interfaces.

```python
# HTML QUICK-STYLER - ADVANCED VERSION (FIXED)
# ==============================================================================

class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
                'accent': '#f093fb',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'vibrant': {
                'primary': '#ff6b6b',
                'secondary': '#ff8e72',
```

Code reference (hint: click on image and you can copy paste from Comment

```python
        'creative': {
            'primary': '#9b59b6',
            'secondary': '#8e44ad',
            'accent': '#e74c3c',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        },
        'minimal': {
            'primary': '#2c3e50',
            'secondary': '#34495e',
            'accent': '#3498db',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        }
    }

    return palettes.get(style.lower(), palettes['modern'])
```

Code reference (hint: click on image and you can copy paste from Comment)

Activity 2.2: HTML Page Builder Implementation

The HTMLPageBuilder class includes:

- Section extraction from user descriptions
- Keyword detection for hero, features, about, portfolio, testimonials, pricing, contact, and footer sections
- SEO meta tags generation with Open Graph and Twitter Card support

Individual section HTML generation with professional template

```
[ ]

class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extract_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }

        desc_lower = description.lower()
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
    @staticmethod
    def generate_meta_tags(title: str, description: str) -> str:
        """Generate SEO meta tags"""
        meta_tags = f'''    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{title}</title>
    <meta name="description" content="{description[:160]}">
    <meta name="theme-color" content="#0f172a">
    <meta property="og:title" content="{title}">
    <meta property="og:description" content="{description[:160]}">
    <meta property="og:type" content="website">
    <meta name="twitter:card" content="summary_large_image">
    <meta name="author" content="HTML Quick-Styler">'''
        return meta_tags

    @staticmethod
    def generate_section_html(section_type: str, palette: Dict) -> str:
        """Generate HTML for specific section"""
        sections = {
            'hero': f'''        <section class="hero">
            <div class="hero-content">
                <h1>Welcome to Your Site</h1>
                <p>Create amazing web experiences</p>
                <button class="btn btn-primary">Get Started</button>
            </div>
        </section>''',

            'features': f'''        <section class="features">
            <h2>Features</h2>
            <div class="features-grid">
                <div class="feature-card">
                    <h3>Fast</h3>
                    <p>Lightning-quick performance</p>
                </div>
                <div class="feature-card">
                    <h3>Secure</h3>
                    <p>Enterprise-grade security</p>
                </div>
                <div class="feature-card">
                    <h3>Scalable</h3>
                    <p>Grow without limits</p>
                </div>
            </div>
        </section>''',

            'about': f'''        <section class="about">
            <div class="about-content">
                <h2>About Us</h2>
                <p>We're passionate about creating beautiful, functional websites that help your business grow.</p>
            </div>
        </section>''',

            'portfolio': f'''        <section class="portfolio">
            <h2>Our Work</h2>
            <div class="portfolio-grid">
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
            </div>
        </section>''',
```

Code reference (hint: click on image and you can copy paste from the comment)

Activity 2.3: CSS Generation Engine

The CSS generator creates comprehensive styling including:

- CSS variables for dynamic color management
- Gradient backgrounds and modern animations
- Responsive grid layouts with Flexbox and CSS Grid
- Smooth transitions and hover effects
- Mobile-first media queries for tablets and phones
- Sticky header with backdrop blur effect

Card-based components with depth and shadows

```python
    @staticmethod
    def generate_css(palette: Dict) -> str:
        """Generate comprehensive CSS"""
        css = f'''    <style>
        * {{
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }}

        :root {{
            --primary: {palette['primary']};
            --secondary: {palette['secondary']};
            --accent: {palette['accent']};
            --background: {palette['background']};
            --surface: {palette['surface']};
            --text-primary: {palette['text_primary']};
            --text-secondary: {palette['text_secondary']};
            --border: {palette['border']};
        }}

        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 3: Gradio Interface Development

Activity 3.1: Main Interface with Multiple Tabs

The Gradio interface includes 5 tabs

```
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, var(--background) 0%, var(--surface) 100%);
    color: var(--text-primary);
    line-height: 1.6;
    min-height: 100vh;
}}

header {{
    background: rgba(15, 23, 42, 0.95);
    backdrop-filter: blur(10px);
    padding: 1.5rem 2rem;
    position: sticky;
    top: 0;
    z-index: 100;
    border-bottom: 1px solid var(--border);
}}

header nav {{
    display: flex;
    justify-content: space-between;
    align-items: center;
    max-width: 1200px;
    margin: 0 auto;
}}

header h1 {{
    font-size: 1.8rem;
    background: linear-gradient(135deg, var(--primary), var(--accent));
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}}

header nav a {{
    color: var(--text-secondary);
    text-decoration: none;
    margin: 0 1.5rem;
    transition: color 0.3s ease;
}}

header nav a:hover {{
    color: var(--primary);
}}

main {{
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 2rem;
}}

section {{
    padding: 4rem 0;
    animation: fadeIn 0.6s ease-in;
}}

@keyframes fadeIn {{
    from {{
        opacity: 0;
        transform: translateY(20px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}

h1, h2, h3 {{
    color: var(--text-primary);
    margin-bottom: 1rem;
}}
```

Code reference (hint: click on image and you can copy paste from Comment)

```
h1 {{
    font-size: 3rem;
    font-weight: 700;
}}

h2 {{
    font-size: 2rem;
    margin-bottom: 2rem;
    text-align: center;
}}

h3 {{
    font-size: 1.3rem;
}}

.hero {{
    background: linear-gradient(135deg, var(--primary) 0%, var(--secondary) 100%);
    color: white;
    padding: 6rem 0;
    text-align: center;
    min-height: 500px;
    display: flex;
    align-items: center;
    justify-content: center;
}}

.hero h1 {{
    color: white;
    font-size: 4rem;
    margin-bottom: 1rem;
    animation: slideDown 0.8s ease;
}}

.hero p {{
    font-size: 1.5rem;
    margin-bottom: 2rem;
    opacity: 0.95;
    animation: slideUp 0.8s ease;
}}

@keyframes slideDown {{
    from {{
        opacity: 0;
        transform: translateY(-30px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}

@keyframes slideUp {{
    from {{
        opacity: 0;
        transform: translateY(30px);
    }}
    to {{
        opacity: 1;
        transform: translateY(0);
    }}
}}
```

Code reference (hint: click on image and you can copy paste from Comment)

```
.btn {{
    padding: 0.8rem 2rem;
    border: none;
    border-radius: 8px;
    font-size: 1rem;
    cursor: pointer;
    transition: all 0.3s ease;
    font-weight: 600;
}}

.btn-primary {{
    background: var(--primary);
    color: white;
    box-shadow: 0 4px 15px rgba(0,0,0,0.3);
}}

.btn-primary:hover {{
    transform: translateY(-2px);
    box-shadow: 0 6px 20px rgba(0,0,0,0.4);
    background: var(--accent);
}}

.btn-secondary {{
    background: var(--secondary);
    color: white;
}}

.btn-secondary:hover {{
    background: var(--primary);
}}

.features-grid {{
    display: grid;
    grid-templ[illegible]lumns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 2rem;
}}

.feature-card {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-radius: 12px;
    border: 1px solid var(--border);
    transition: all 0.3s ease;
    backdrop-filter: blur(10px);
}}

.feature-card:hover {{
    transform: translateY(-8px);
    border-color: var(--primary);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}}

.feature-card h3 {{
    color: var(--primary);
    margin-bottom: 0.5rem;
}}

.feature-card p {{
    color: var(--text-secondary);
}}
```

Code reference (hint: click on image and you can copy paste from Comment)

```
.portfolio-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 2rem;
}}

.portfolio-item {{
    aspect-ratio: 1;
    background: linear-gradient(135deg, var(--primary), var(--secondary));
    border-radius: 12px;
    cursor: pointer;
    transition: all 0.3s ease;
    border: 1px solid var(--border);
}}

.portfolio-item:hover {{
    transform: scale(1.05);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}}

.about {{
    text-align: center;
}}

.about-content {{
    max-width: 800px;
    margin: 0 auto;
}}

.about p {{
    font-size: 1.1rem;
    color: var(--text-secondary);
}}

.testimonials-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 2rem;
}}

.testimonial {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-left: 4px solid var(--primary);
    border-radius: 8px;
}}

.testimonial p {{
    font-style: italic;
    margin-bottom: 1rem;
    color: var(--text-secondary);
}}

.testimonial span {{
    font-weight: 600;
    color: var(--primary);
}}

.pricing-grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: 2rem;
}}
```

Code reference (hint: click on image and you can copy paste from Comment)

```
.pricing-card {{
    background: rgba(30, 41, 59, 0.8);
    padding: 2rem;
    border-radius: 12px;
    border: 1px solid var(--border);
    text-align: center;
    transition: all 0.3s ease;
}}

.pricing-card.featured {{
    border-color: var(--primary);
    box-shadow: 0 20px 40px rgba(0,0,0,0.3);
    transform: scale(1.05);
}}

.pricing-card .price {{
    font-size: 2.5rem;
    color: var(--primary);
    margin: 1rem 0;
    font-weight: 700;
}}

.pricing-card ul {{
    list-style: none;
    margin: 1.5rem 0;
    text-align: left;
}}

.pricing-card li {{
    padding: 0.5rem 0;
    color: var(--text-secondary);
}}

.contact {{
    max-width: 600px;
    margin: 0 auto;
}}

.contact-form {{
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
}}

.contact-form input,
.contact-form textarea {{
    padding: 1rem;
    border: 1px solid var(--border);
    border-radius: 8px;
    background: rgba(30, 41, 59, 0.8);
    color: var(--text-primary);
    font-family: inherit;
    font-size: 1rem;
}}

.contact-form input:focus,
.contact-form textarea:focus {{
    outline: none;
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(0,0,0,0.3);
}}
```

Code reference (hint: click on image and you can copy paste from Comment)

```
        .footer-content {{
            max-width: 1200px;
            margin: 0 auto;
            padding: 0 2rem;
            display: flex;
            justify-content: space-between;
            align-items: center;
            flex-wrap: wrap;
            gap: 2rem;
        }}

        .footer-links {{
            display: flex;
            gap: 2rem;
        }}

        .footer-links a {{
            color: var(--text-secondary);
            text-decoration: none;
            transition: color 0.3s ease;
        }}

        .footer-links a:hover {{
            color: var(--primary);
        }}

        @media (max-width: 768px) {{
            header nav {{
                flex-direction: column;
                gap: 1rem;
            }}

            header h1 {{
                font-size: 1.4rem;
            }}

            .hero h1 {{
                font-size: 2rem;
            }}

            .hero p {{
                font-size: 1.1rem;
            }}

            h2 {{
                font-size: 1.5rem;
            }}

            main {{
                padding: 0 1rem;
            }}

            section {{
                padding: 2rem 0;
            }}

            .footer-content {{
                flex-direction: column;
                text-align: center;
            }}
        }}
    </style>'''
    return css
```

```python
class HTMLGenerationEngine:
    """Main engine for HTML page generation"""

    @staticmethod
    def generate_page(title: str, description: str, style: str = "modern") -> Tuple[str, str]:
        """Generate complete HTML page"""

        # Generate color palette
        palette = ColorPaletteGenerator.generate_palette(style)

        # Extract sections
        sections = HTMLPageBuilder.extract_sections(description)

        # Generate meta tags
        meta_tags = HTMLPageBuilder.generate_meta_tags(title, description)

        # Generate sections
        sections_html = '\n\n'.join([
            HTMLPageBuilder.generate_section_html(section, palette)
            for section in sections
            if HTMLPageBuilder.generate_section_html(section, palette)
        ])
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
        # Generate CSS
        css = HTMLPageBuilder.generate_css(palette)

        # Build complete HTML
        html = f'''<!DOCTYPE html>
<html lang="en">
<head>
{meta_tags}
{css}
</head>
<body>
    <header>
        <nav>
            <h1>{title}</h1>
            <div>
                <a href="#features">Features</a>
                <a href="#about">About</a>
                <a href="#contact">Contact</a>
            </div>
        </nav>
    </header>

    <main>
{sections_html}
    </main>
</body>
</html>'''

        return html, palette

generation_history = []

def generate_html_page(title: str, description: str, style: str = "modern") -> Tuple[str, str, str]:
    """Generate HTML page with styling"""
    if not title.strip() or not description.strip():
        return "", "❌ Please provide both title and description!", ""

    try:
        html_output, palette = HTMLGenerationEngine.generate_page(title, description, style)

        color_info = f"""
**Color Palette:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
"""

        generation_history.append({
            'title': title,
            'description': description,
            'html': html_output,
            'palette': palette,
            'timestamp': datetime.now().isoformat()
        })
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
        return html_output, "✅ HTML page generated successfully!", color_info
    except Exception as e:
        return "", f"❌ Error: {str(e)}", ""

def get_color_suggestions(style: str) -> str:
    """Get color suggestions based on style"""
    palette = ColorPaletteGenerator.generate_palette(style)

    palette_text = f"""
**Palette for "{style}" style:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
- Text: {palette['text_primary']}
"""
    return palette_text

print("🚀 Loading HTML Quick-Styler Advanced...")

with gr.Blocks(title="HTML Quick-Styler - Advanced Edition") as demo:

    gr.Markdown("""
    # 🎨 HTML Quick-Styler Advanced
    ## Professional HTML Page Generator with Auto-Styling

    **Generate production-ready HTML pages in seconds**
    """)

    with gr.Tabs():
```

Code reference (hint: click on image and you can copy paste from Comment)

Tab 1: Generate Page

```python
    with gr.Tabs():

        # ============ TAB 1: Page Generator ============
        with gr.TabItem("🎯 Generate Page"):
            with gr.Row():
                with gr.Column(scale=1):
                    gr.Markdown("### 📝 Page Details")

                    page_title = gr.Textbox(
                        label="Page Title",
                        value="Modern Portfolio Website",
                        lines=1
                    )

                    page_description = gr.Textbox(
                        label="Page Description & Sections",
                        value="Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact for
                        lines=4
                    )

                    color_style = gr.Dropdown(
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
            )

            color_style = gr.Dropdown(
                ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                value="modern",
                label="🎨 Color Style"
            )

            generate_btn = gr.Button("✨ Generate HTML", size="lg", variant="primary")

        with gr.Column(scale=1):
            gr.Markdown("### 📊 Output")

            html_output = gr.Code(
                language="html",
                label="Generated HTML/CSS",
                lines=20
            )

            status_msg = gr.Markdown()
            color_info = gr.Markdown()

    generate_btn.click(
        generate_html_page,
        inputs=[page_title, page_description, color_style],
        outputs=[html_output, status_msg, color_info]
    )
```

Tab 2: Live Preview

```python
            outputs=[html_output, status_msg, color_info]
        )

    # ============= TAB 2: Preview =============
    with gr.TabItem("👁️ Live Preview"):
        gr.Markdown("### 🔍 Preview Generated Page")

        preview_button = gr.Button("🔄 Load Preview", size="lg", variant="primary")
        preview_output = gr.HTML(value="<p style='text-align:center; color:#fff; padding:40px;'>No preview loaded. Generate a page first.</p>")

        def show_preview(html_text):
            if not html_text.strip():
                return "<p style='text-align:center; color:#fff; padding:40px;'>No HTML provided</p>"
            return html_text

        preview_button.click(
            show_preview,
            inputs=[html_output],
            outputs=[preview_output]
        )

    # ============= TAB 3: Color Palettes =============
    with gr.TabItem("🎨 Color Palettes"):
        gr.Markdown("### 🌈 Explore Color Schemes")

        with gr.Row():
```

Code reference (hint: click on image and you can copy paste from Comment)

Tab 3: Color Palettes

```python
            outputs=[preview_output]
        )

    # ============= TAB 3: Color Palettes =============
    with gr.TabItem("🎨 Color Palettes"):
        gr.Markdown("### 🌈 Explore Color Schemes")

        with gr.Row():
            style_selector = gr.Radio(
                ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                value="modern",
                label="Choose Style"
            )

            palette_btn = gr.Button("🎨 Get Palette", size="lg", variant="primary")

        palette_display = gr.Markdown()

        palette_btn.click(
            get_color_suggestions,
            inputs=[style_selector],
            outputs=[palette_display]
        )
```

Code reference (hint: click on image and you can copy paste from Comment)

Tab 4: Templates

```python
# ============= TAB 4: Templates =============
with gr.TabItem("📋 Templates"):
    gr.Markdown("### 📝 Quick Templates")

    with gr.Row():
        with gr.Column():
            gr.Markdown("""
            #### Portfolio Website
            Hero, Portfolio Gallery, About, Contact, Footer
            """)
            portfolio_btn = gr.Button("Use Template", size="sm")

            def load_portfolio():
                return ("Portfolio Website",
                        "Hero section with welcome banner, portfolio gallery with 6 items, about section, testimonials, contact form, foot
                        "modern")

            portfolio_btn.click(load_portfolio, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### SaaS Landing Page
            Hero, Features, Pricing, Testimonials, CTA, Footer
            """)
            saas_btn = gr.Button("Use Template", size="sm")
```

Code reference (hint: click on image and you can copy paste from Comment)

```python
            saas_btn = gr.Button("Use Template", size="sm")

            def load_saas():
                return ("SaaS Product",
                        "Hero banner, features showcase with 3 items, pricing plans comparison, testimonials, call-to-action button, footer",
                        "vibrant")

            saas_btn.click(load_saas, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### Company Website
            Header, About, Services, Team, Contact, Footer
            """)
            company_btn = gr.Button("Use Template", size="sm")

            def load_company():
                return ("Company",
                        "Header with navigation, hero section, about company, services list, team members, contact form, footer",
                        "ocean")

            company_btn.click(load_company, outputs=[page_title, page_description, color_style])
```

Code reference (hint: click on image and you can copy paste from Comment)

Tab 5: Guide

```python
# ============= TAB 5: Guide =============
with gr.TabItem("📚 Guide"):
    gr.Markdown("""
    # 📖 HTML Quick-Styler Guide

    ## How It Works

    ### 1. Enter Details
    - **Title:** Page name
    - **Description:** Sections you want
    - **Color Style:** Pick a theme

    ### 2. Automatic Detection
    The tool recognizes these keywords:
    - `hero` / `banner` → Hero Section
    - `features` / `services` → Features Grid
    - `about` / `team` → About Section
    - `portfolio` / `gallery` → Portfolio
    - `testimonial` / `reviews` → Testimonials
    - `pricing` / `plans` → Pricing Plans
    - `contact` / `form` → Contact Form
    - `footer` → Footer
```

Code reference (hint: click on image and you can copy paste from Comment)

MILESTONE 4: Deployment

Activity 4.1: Google Colab Deployment

Launch the application in Google Colab:

```
print("\n✅ HTML Quick-Styler Advanced Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

Code reference (hint: click on image and you can copy paste from Comment)

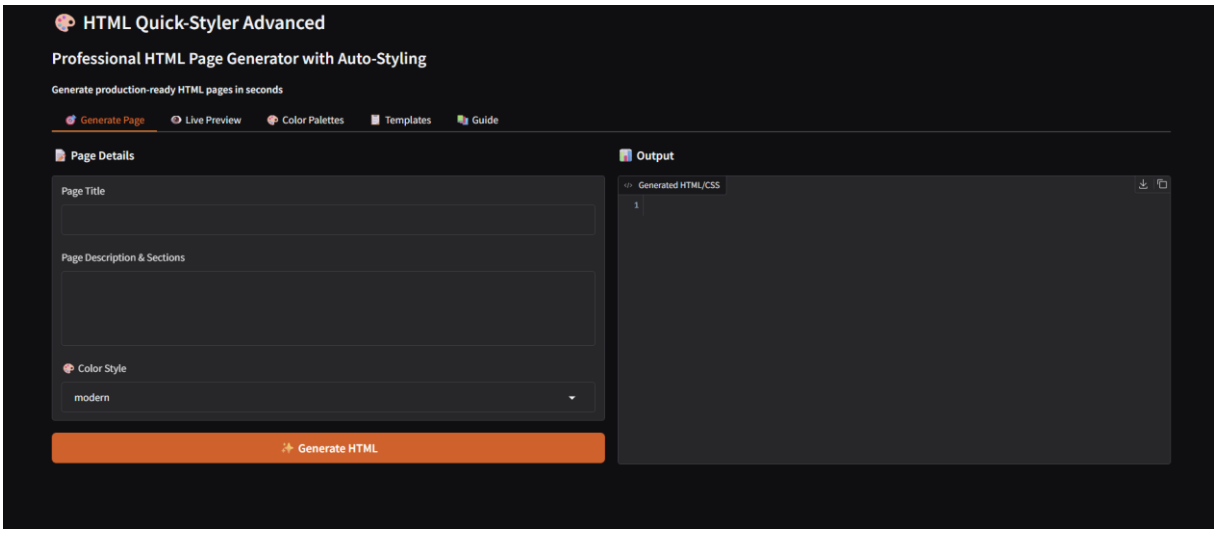
```
🚀 Loading HTML Quick-Styler Advanced...
ut actions
L Quick-Styler Advanced Ready!

/tmp/ipython-input-1159347522.py:956: DeprecationWarning: The 'show_api' parameter in launch() will be removed
  demo.launch(share=True, show_error=True, show_api=False)
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://87205c7f5a55c5adc8.gradio.live
```

Code reference (hint: click on image and you can copy paste from Comment)

The application will be accessible via:

- Public URL: `: https://87205c7f5a55c5adc8.gradio.live`



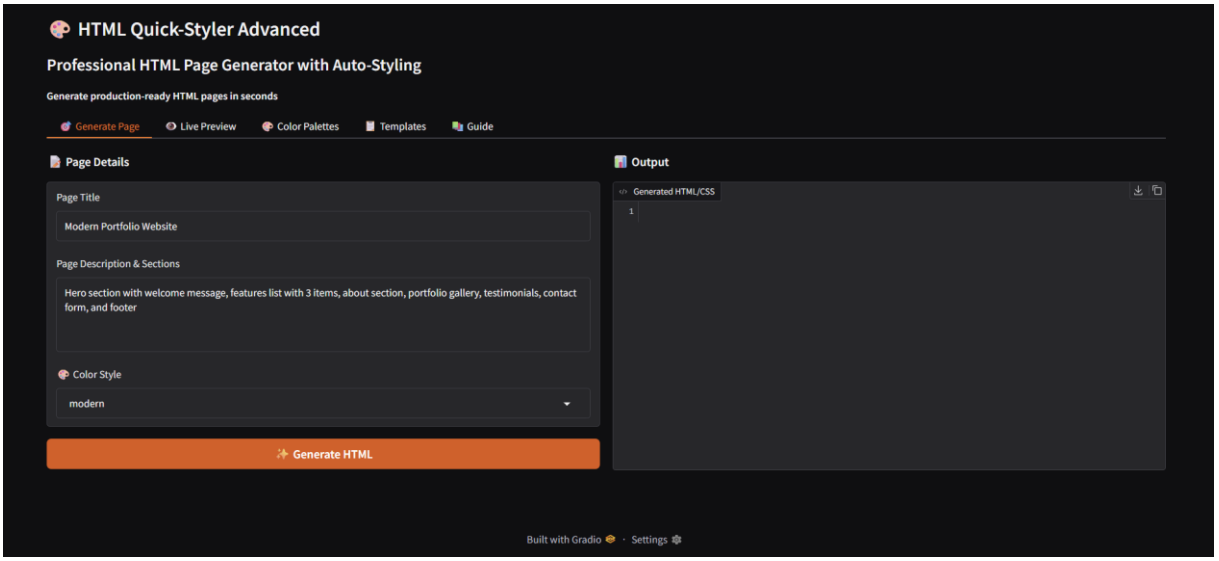
MILESTONE 5: Testing & Validation

Activity 5.1: Functional Testing

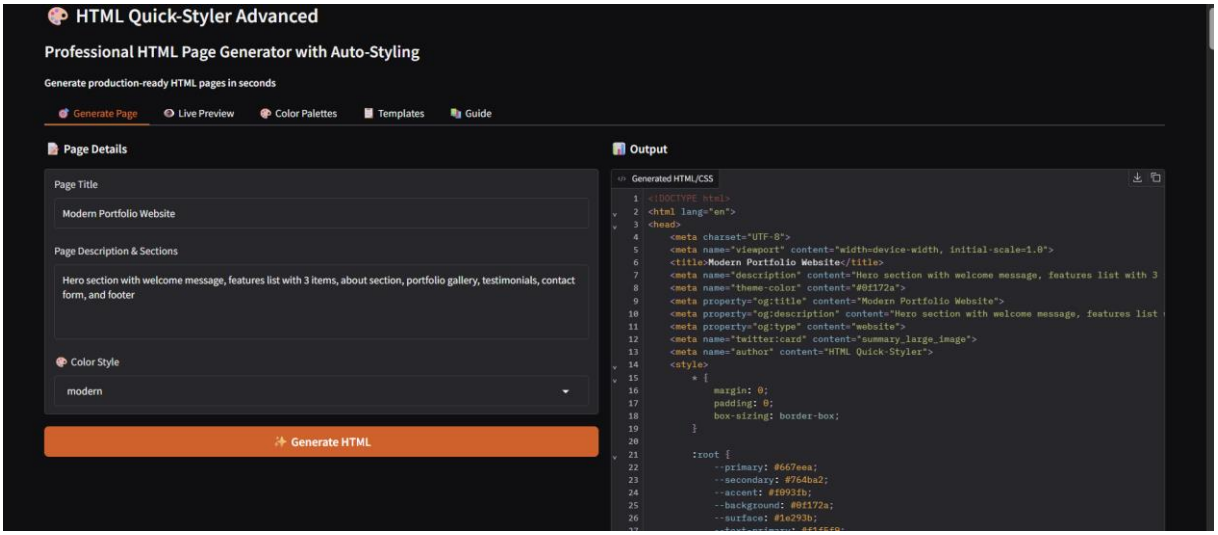
Test Case 1: Portfolio Website Generation

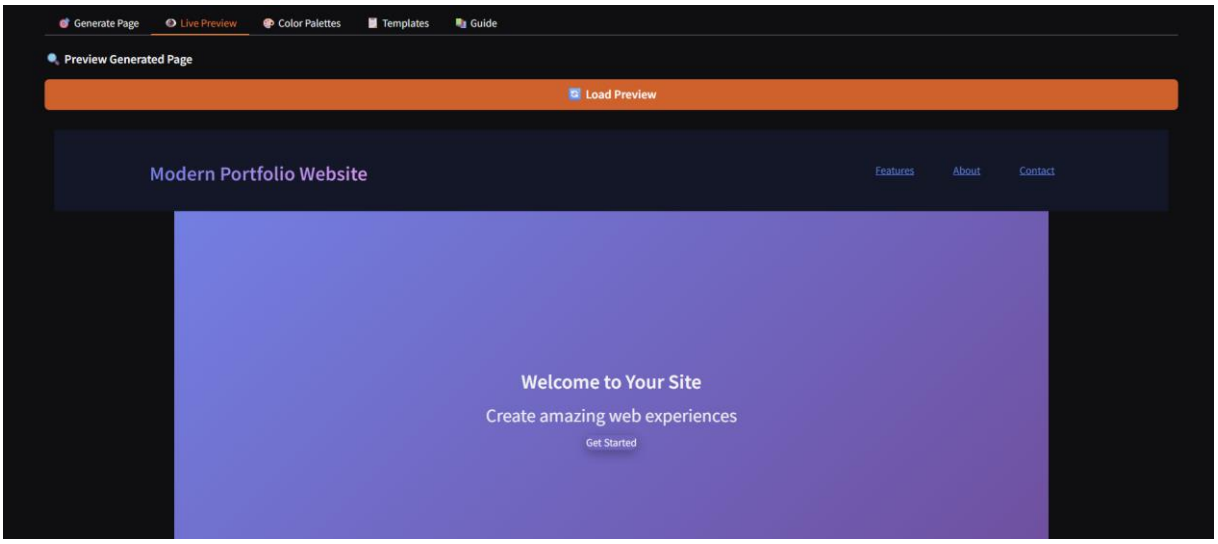
Input:

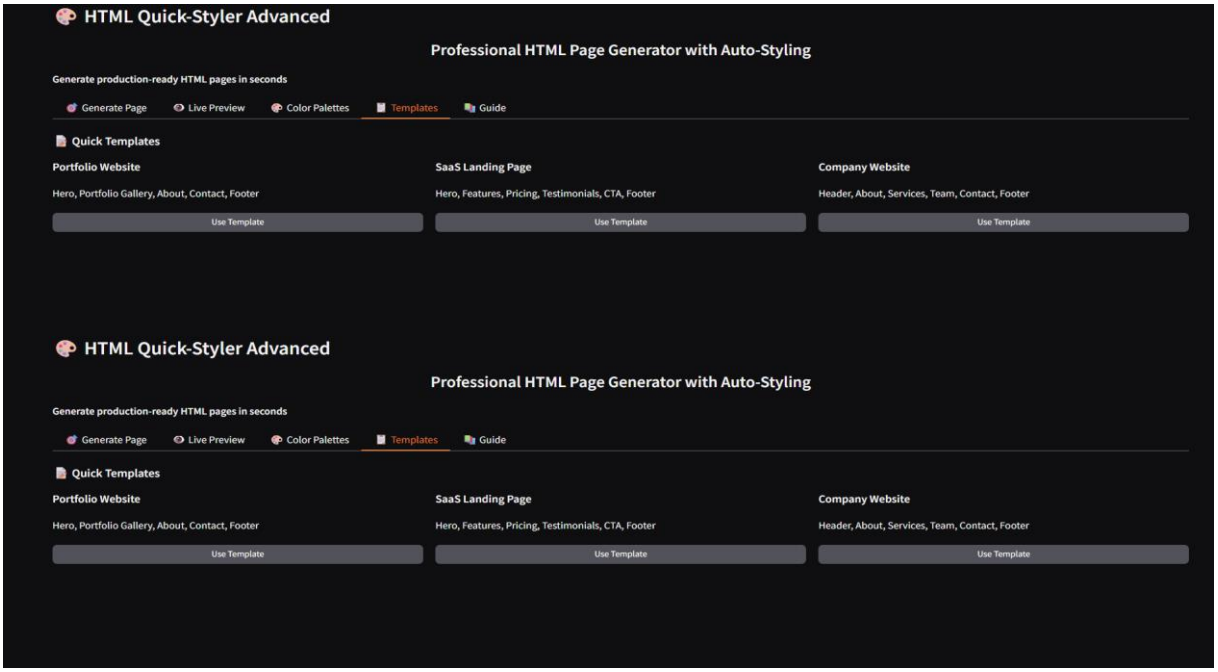
- Title: Modern Portfolio Website
- Description: Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact form, and footer
- Color Style: modern

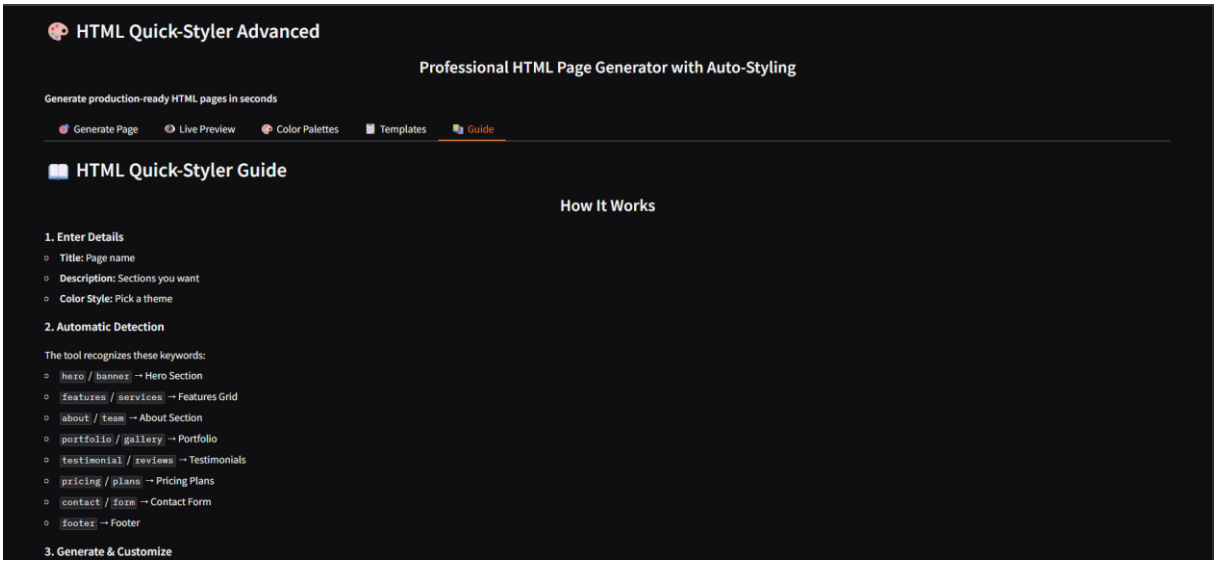


Expected Output:









Conclusion

HTML Quick-Styler successfully delivers an intelligent web page generation assistant that combines intuitive design templates with IBM Granite 3.3 2B Instruct model capabilities. By leveraging Gradio's powerful interface framework and Google Colab's cloud infrastructure, the system provides fast, professional, and customizable HTML/CSS generation accessible to designers, developers, and non-technical users alike.

The platform demonstrates the viability of AI-assisted web development, making professional page creation accessible while maintaining high design standards, responsive layouts, and

production-ready code quality. With eight distinct color palettes, intelligent section detection, and comprehensive template options, HTML Quick-Styler empowers users to create beautiful websites in minutes rather than hours.

Future Enhancement

Future enhancements could include custom color picker integration, JavaScript interactivity generation, component library expansion, and integration with popular web frameworks. The modular architecture allows for easy extension and customization to meet evolving user needs and design trends.The platform can also be expanded to support multi-page website generation, including pages such as Home, About, Services, Blog, and Contact with automatic navigation linking. Integration with modern web frameworks like React, Vue, or Bootstrap can help developers directly use the generated designs in real-world applications. Additionally, built-in SEO optimization, accessibility improvements, and performance enhancements can ensure that generated websites follow modern web standards. Future updates may also include a larger template library for different industries, real-time preview and live editing features, AI-based content generation, and direct deployment options to hosting platforms, making the system a complete solution for rapid and intelligent web development.